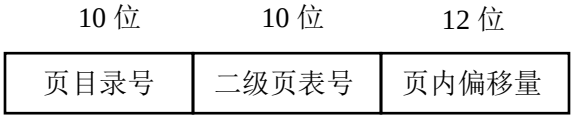
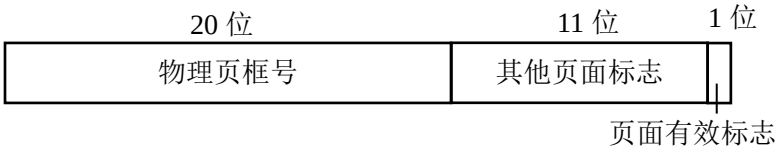
1. 一个 32 位的页式虚拟存储系统采用两级页表管理，其逻辑地址中，第 22 到 31 位是第一级页表（页目录）索引，第 12 位到 21 位是第二级页表索引，页内偏移占第 0 到 11 位。每个页表（目录）项包含 20 位物理页框号和 12 位标志位，其中最后 1 位为页面有效标志。

逻辑地址格式：



页目录项、页表项格式：



（1）请问进程整个的地址空间有多少字节？一页有多少字节？

进程整个的地址空间为 2^32=4G 字节，一页有 2^12=4K 字节。

（2）一个进程如果从 0x80000000 开始映射 4MB 大小页表空间，求第一级页表（页目录）的起始逻辑地址，并指出从哪个逻辑地址可以读出第一级页表（页目录）所在的物理页框号。说明理由。（注意 B 代表字节，一个 32 位地址占 4 字节）

页目录的起始地址是 0x80000000，将其右移 10 位可得到 0x80200000。由此可以读出第一级页表（即页目录）所在的物理页框号的逻辑地址。页目录自映射表项地址的计算方式如下：0x80000000 | 0x80000000 >> 10 | 0x80000000 >> 20，结果为 0x80200800。

（3）如果当前进程的页目录物理基地质、页目录和相应页表内容如图下所示，请描述访问以下逻辑地址时系统进行地址转换的过程，如可行给出最终访存获取到的数据。逻辑地址：0x0、0x00803004、0x00402001

0x0：虚拟页目录 0，查询对应 0 号页目录项，得到页目录项 0x0，由于有效标志位为 0，表示该 页面无效，因此页表中不存在该虚拟地址到物理地址的映射，产生缺页中断。 0x00803004：虚拟页目录索引号为 2，二级页表索引号为 3，页内偏移为 4，查询对应 2 号页目 录项，得到 0x5001，有效标志位为 1，在物理地址 0x5000 查找二级页表，找到对应的 3 号页表 项 0x20001，由于有效标志位为 1，对应物理页框基地址为 0x20000，根据页内偏移 4，查找页 内偏移为 4 的地址，按字寻址得到数据 0x326001，按字节寻址得到数据 0x00 或 0x01。 0x00402001：虚拟页目录索引号为 1，二级页表索引号为 2，页内偏移为 1，查询对应 1 号页目 录项，得到 0x1001，有效标志位为 1，在物理地址 0x1000 查找二级页表，即为页目录自身，找 到对应的 2 号页表项 0x5001，由于有效标志位为 1，对应物理页框基地址为 0x5000，根据页内 偏移 1，查找对应内存，得到数据 0x00。

（4）要想访问物理地址 0x326028，需要使用哪个逻辑地址？

要想访问物理地址 0x326028，需要使用虚拟地址 0x00C01028，其中虚拟页目录索引号为 3，二 级页表索引号为 1，页内偏移为 0x028

页目录物理基地址

0x1000

页目录
物理地址：0x1000

0	0x0
1	0x1001
2	0x5001
3	0x20001
4	0x0
	…
1023	0x0

页表
物理地址：0x5000

0	0x0
1	0x4e001
2	0x67001
3	0x20001
4	0x0
	…
1023	0x0

页表
物理地址：0x20000

0	0x9000
1	0x326001
2	0x41001
3	0x0
4	0x0
	…
1023	0x0

2. 一个 32 位的虚拟存储系统采用两级页表管理，其逻辑地址形式如下：

第一级页表（10 位）	第二级页表（10 位）	页内偏移（12 位）

物理地址为 32 位，形式为：

物理页框号（20 位）	页内偏移（12 位）

页表项（PTE）格式为：

高 20 位	低 12 位（为标志位）
物理页框号	第 0 位为有效位，如果为 0，表示 Invalid；如果为 1 表示 Valid。第 1 位为读写位，如果为 0，表示 Read Only ；如果为 1 表示 Read/Write。

假设当前进程第一级页表的物理地址为 0x00200000，利用后面物理内存的信息，请在下表中写出以下指令的执行结果。

对于 Load 指令，如果成功执行，写出读入的数据（读取一个字节），否则写 Error；

对于 Store 指令，如果成功执行，写 OK，否则写 Error。并给出寻址过程。

指令	结果
Load [0x00001022]	20
Store [0x00C07222]	Error:read-only
Store[0x00C005BF]	ok
Load [0x00003013]	0x19
Load [0xFF80078F]	Error:Invaild
Load [0xFFFFF005]	0x66

物理内存（大尾端）

Address	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
00000000	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
00000010	1E	1F	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D
….																
00001010	40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
00001020	40	03	41	01	30	01	31	03	00	03	00	00	00	00	00	00
00001030	00	11	22	33	44	55	66	77	88	99	AA	BB	CC	DD	EE	FF
00001040	10	01	11	03	31	03	13	00	14	01	05	03	16	01	17	00
….																
00002030	10	01	11	00	12	03	67	03	11	03	00	00	00	00	00	00
00002040	02	20	03	30	04	40	05	50	01	60	03	70	08	80	09	90
00002050	10	00	31	01	10	03	31	01	12	03	30	00	10	00	10	01
….																
00004000	30	00	31	01	11	01	33	03	34	01	35	00	43	38	32	79
00004010	50	28	84	19	71	69	39	93	75	10	58	20	97	49	44	59
00004020	23	03	20	03	00	01	62	08	99	86	28	03	48	25	34	21

….																
00100000	00	00	10	65	00	00	40	67	00	00	30	00	00	00	40	07
00100010	00	00	50	03	00	00	00	00	00	00	00	00	00	00	00	00
….																
00103000	11	22	00	67	55	66	77	88	99	AA	BB	CC	DD	EE	FF	00
00103010	22	33	44	55	66	77	88	99	AA	BB	CC	DD	EE	FF	00	01
….																
001FE000	04	15	00	00	48	59	70	7B	8C	9D	AE	BF	D0	E1	F2	03
001FE010	10	15	00	67	10	15	10	67	10	15	20	67	10	15	30	67
….																
001FF000	00	00	00	00	00	00	00	65	00	00	10	67	00	00	00	00
001FF010	00	00	20	67	00	00	30	67	00	00	40	67	00	00	50	07
….																
001FFFF0	00	00	00	00	00	00	00	00	10	00	00	67	00	10	30	67
….																
00200000	00	10	00	07	00	10	10	07	00	10	20	07	00	10	30	07
00200010	00	10	40	07	00	10	50	07	00	10	60	07	00	10	70	07
00200020	00	10	00	07	00	00	00	00	00	00	00	00	00	00	00	00
….																
00200FF0	00	00	00	00	00	00	00	00	00	1F	E0	07	00	1F	F0	07
….																